

Spring AI 2.0 Cheat Sheet

Irurueta Docs · Guides & References · AI · Spring AI

Baseline: Spring AI 2.0.1 · Spring Boot 4.0/4.1 (4.1.1) · Java 17+ (examples Java 21) · verified 2026-09-29. Jackson 3 (tools, jackson), JSpecify. MCP skew: Spring AI 2.0.x's MCP Java SDK (2.0.0) targets spec 2025-11-25, while this site's MCP baseline is 2026-07-28 (stateless core, server/discover not exposed yet). Quick reference only — verify against the official docs before production use.

STARTERS & BOM

<dependencyManagement> import org.springframework.ai:spring-ai-bom:2.0.1 parent: spring-boot-starter-parent 4.1.1 web: spring-boot-starter-webmvc

Artifact pattern	Examples
spring-ai-starter-model-<provider>	ollama, openai, anthropic, google-genai, mistral-ai, deepseek, bedrock-converse
...-starter-vector-store-<store>	pgvector, redis, qdrant, chroma
...-starter-mcp-<role>	mcp-client, -client-webflux, mcp-server, -server-webmvc, -server-webflux
...-starter-model-chat-memory-<repository>-<store>	jdbc, cassandra, mongodb, neo4j, redis
spring-ai-starter-tool-search-advisor	tool search (regex lucene vector)
Not starters	spring-ai-vector-store-advisor, spring-ai-rag, spring-ai-spring-boot-docker-compose, spring-ai-spring-boot-testcontainers

Old form spring-ai-<...>-spring-boot-starter is gone. No Azure OpenAI starter: use the OpenAI one. Only the BOM sets versions.

KEY 2.0 PROPERTIES (FLATTENED)

spring.ai.model.chat-ollama # also .embedding | .image | .moderation # .audio.speech | .audio.transcription spring.ai.ollama.base-url=http://localhost:11434 spring.ai.ollama.chat.model=llama3.1:8b spring.ai.ollama.embedding.model=nomic-embed-text spring.ai.ollama.init.pull-model-strategy=when_missing spring.ai.openai.api-key=\${OPENAI_API_KEY} spring.ai.openai.chat.model=... spring.ai.openai.base-url=http://host:8000/v1

- No .options segment: spring.ai.openai.chat.model, not ...chat.options.model (old form = deprecated alias). ollama.chat.think-option → think
- OpenAI-compatible local servers: base-url must include /v1 (used as is)
- No forced temperature 0.7: provider default applies
- Two starters, no selector = two beans: set spring.ai.model.*
- spring.ai.retry.max-attempts (default 10), backoff.initial-interval | multiplier | max-interval: RestClient providers only; OpenAI/Anthropic use spring.ai.openai.max-retries (3), .timeout (60s)
- spring.ai.chat.client.tool-calling.enabled | advisor-order
- spring.ai.tools.limits.max-total-tool-calls | max-calls-per-tool, .throw-exception-on-error, .resolution, fallback.enabled
- spring.ai.chat.memory.repository.jdbc.initialize-schema=always

CHATCLIENT IDIOMS

@Bean ChatClient docs(ChatClient.Builder b, DocsTools t) { return b.defaultSystem("You are {role}.") .defaultAdvisors(new SimpleLoggerAdvisor()) .defaultOptions(ChatOptions.builder() .temperature(0.2).maxTokens(600)) // no build() .defaultTools(t) } .defaultToolContext(Map.of("tenant", "public")) .build(); }

String a = client.prompt() .system(s -> s.param("role", "DocsAssistant")) .user(question) .advisors(s -> s.param(ChatMemory.CONVERSATION_ID, id)) .call().content(); // String .call().entity(DocMetadata.class); // record .call().chatResponse(); // usage, metadata Flux<String> f = client.prompt().user(q) .stream().content(); // or .chatResponse() client.mutate().defaultSystem("...").build();

- ChatModel.call(new Prompt(q)) = low level; ChatClient = fluent, advisors, tools
- .options(...)/.defaultOptions(...) take the builder; request options replace defaults (no merge)
- Escape {x} literals in templates; streaming needs the reactive stack even under MVC

ADVISOR ORDER (LOWEST RUNS FIRST)

Advisor / constant	Order
Ordered.HIGHEST_PRECEDENCE	Integer.MIN_VALUE
CE	
Your input guard, e.g. SafeGuardAdvisor*	MIN + 50
Advisor.DEFAULT_CHAT_MEMORY_PRECEDENCE_ORDER (MessageChatMemoryAdvisor)	MIN + 200
ToolCallingAdvisor.DEFAULT_LT_ORDER	MIN + 300
SimpleLoggerAdvisor, SafeGuardAdvisor, QuestionAnswerAdvisor, VectorStoreChatMemoryAdvisor (MIN+200)	0 (default)
StructuredOutputValidationAdvisor	MAX - 2000
ChatModelCallAdvisor/StreamAdvisor	LOWEST_PRECEDENCE

- Types CallAdvisor, StreamAdvisor (...client.advisor.api); equal orders = unspecified: give each a distinct value
- Memory outside the tool loop (+200 < +300): only the final user/assistant exchange is stored, never tool messages
- Order 0 advisors sit inside the tool loop: RAG advisors re-retrieve on every pass; use MIN+250. *SafeGuardAdvisor at 0 scans history: one blocked message blocks the chat; set MIN+50

@TOOL & TOOL-CALLBACK TYPES

@Tool(description = "Search the docs") public List<DocHit> searchDocs(@ToolParam(description = "query") String q, @ToolParam(required = false) String component, ToolContext ctx) { ... } // ctx.getContext()

client.prompt().user(q).tools(docsTools) // per request .toolContext(Map.of("requester", user))

Type	Use
ToolCallbacks.from(obj)	all @Tool methods → ToolCallback[]
MethodToolCallbackProvider.builder().toolObjects(o).build()	ToolCallbackProvider (bean or .tools())
MethodToolCallback	one method, custom definition
FunctionToolCallback.builder(name, fn).description().inputType().build()	a Function bean/lambda
ToolCallingChatOptions	options carrying tools

- ToolCallingAdvisor (was ToolCallAdvisor) owns the loop, auto-registered; off: spring.ai.chat.client.tool-calling.enabled=false or AdvisorParams.toolCallingAdvisorAutoRegister(false); ToolExecutionEligibilityChecker; markers ToolAdvisor, MemoryAdvisor
- ToolSearchToolCallingAdvisor (...advisor.toolsearch) for big catalogues: spring.ai.chat.client.tool-search-advisor.enabled | tool-index-type | max-results
- 2.0.1: only tools attached to the request run (bean fallback off; unknown tool = IllegalStateException (fails closed); @PreAuthorize on @Tool works, streaming needs spring.reactor.context-propagation=auto

CHAT MEMORY SETUP

@Bean ChatMemory mem(ChatMemoryRepository r) { return MessageWindowChatMemory.builder() .chatMemoryRepository(r).maxMessages(20).build(); }

builder.defaultAdvisors(MessageChatMemoryAdvisor .builder(chatMemory).build());

.advisors(s -> s.param(ChatMemory.CONVERSATION_ID, id)) // REQUIRED chatMemory.clear(id);

- Conversation ID is mandatory (no DEFAULT_CONVERSATION_ID); derive it server-side from the principal, e.g. name + ":" + session
- MessageWindowChatMemory counts system messages; in-memory repo unless a starter is present. PromptChatMemoryAdvisor removed → MessageChatMemoryAdvisor
- Repositories: JDBC (conversation_id VARCHAR(36); hash to a UUID; sequence_id column), Cassandra, MongoDB, Neo4j, Redis (...repository, redis); Cosmos DB removed
- Memory inside the loop: disableInternalConversationHistory(). Cancelled stream stores the user message only. Community spring-ai-session planned to replace ChatMemory in 2.1

MCP CLIENT & SERVER

client (spring-ai-starter-mcp-client) spring.ai.mcp.client.type=SYNC # | ASYNC spring.ai.mcp.client.request-timeout=30s spring.ai.mcp.client.initialized=false # lazy spring.ai.mcp.client.streamable-http.connections .github.url=https://api.githubcopilot.com .github.endpoint=/mcp/readonly spring.ai.mcp.client.studio.connections.x.com and spring.ai.mcp.client.studio.servers-configuration=classpath:mcp-servers.json # sse.connections too

server (spring-ai-starter-mcp-server-webmvc) spring.ai.mcp.server.protocol=STREAMABLE # | STATELESS | SSE; endpoint /mcp spring.ai.mcp.server.name | version | instructions spring.ai.mcp.server.streamable-http.mcp-endpoint spring.ai.mcp.server.capabilities.tool/resource/prompt spring.ai.mcp.server.annotation-scanner.enabled

- Taken from env, never literal: \${GITHUB_MCP_TOKEN} (expanded in properties, not in mcp-servers.json)
- Client beans: McpClientCustomizer<HttpClientStreamableHttpTransport.Builder> (per connection name; e.g. Authorization header), McpClientCustomizer<McpClient.SyncSpec>, McpToolFilter (filters the provider only), SyncMcpToolCallbackProvider
- Server: any ToolCallbackProvider bean becomes MCP tools; SDK 2.0 validates tool input by default (validateToolInputs(false))
- Annotations (org.springframework.ai.mcp.annotation, in spring-ai-mcp-annotations): @McpTool (+McpAnnotations.readOnlyHint...), @McpToolParam, @McpResource, @McpPrompt, @McpArg, @McpComplete; client side @McpLogging, @McpSampling, @McpElicitation, @McpProgress
- Method params: McpSyncRequestContext (skipped in STATELESS) or McpTransportContext
- Security (Spring AI Community 0.1.14): mcp-server-security, mcp-client-security; McpServerOAuth2Configurer.mcpServerOAuth2(); @PreAuthorize on @McpTool; set allowed origins/hosts

AUDIO APIS

spring.ai.model.audio.transcription=openai spring.ai.model.audio.speech=openai | elevenlabs spring.ai.openai.audio.transcription.model=gpt-transcribe spring.ai.openai.audio.speech.model=gpt-4o-mini-tts

String t = transcriptionModel.transcribe(res); Flux<String> s = transcriptionModel.streamTranscribe(res); byte[] mp3 = ttsModel.call("Hello"); ttsModel.call(new TextToSpeechPrompt(text, TextToSpeechOptions.builder().voice("nova") .format("opus") .speed(1.1).build())); Flux<byte[]> f = ttsModel.stream(text);

- TTS = TextToSpeechModel (+ StreamingTextToSpeechModel, TextToSpeechPrompt, TextToSpeechResponse; org.springframework.ai.audio.tts). Pre-2.0 docs/books call it SpeechModel: not present in 2.0.1
- TranscriptionModel extends StreamingTranscriptionModel; whisper-1 does not stream
- Provider option builders pre-fill the model: start from builder().from(model.getOptions())

IMAGE API

spring.ai.model.image=openai # stabilityai/google-genai spring.ai.openai.image.model=gpt-image-2.5-flare spring.ai.openai.timeout=150s # default 60s

ImageResponse r = imageModel.call(new ImagePrompt(text, ImageOptionsBuilder.builder() .width(1024).height(1024).n(1).build()); Image img = r.getResult().getOutput(); img.getB64Json(); img.getUrl(); // one is set OpenAiImageOptions.builder().model(..).quality("medium")

Starters: -model=openai, -stability-ai, -google-genai-image; ModerationModel for moderation

EVALUATOR SNIPPETS

// org.springframework.ai.chat.evaluation var rel = RelevancyEvaluator.builder() .chatClientBuilder(judgeBuilder).build(); var fc = FactCheckingEvaluator .builder(judgeBuilder).build(); EvaluationResponse r = rel.evaluate(new EvaluationRequest(question, docs, answer)); // docs = List<Document> r.isPass(); r.getScore(); r.getFeedback();

- Evaluator is in org.springframework.ai.evaluation; pass = judge replies exactly "yes"; FactChecking score is 0 even on pass (read isPass()); use the builder

OBSERVABILITY NAMES

Micrometer	Prometheus
gen_ai.client.operation (per model call)	gen_ai_client_operation_seconds_{count, sum, max, bucket}
gen_ai.client.token.usage (gen_ai.token.type = input/output/total)	gen_ai_client_token_usage_total
spring.ai.chat.client	spring_ai_chat_client_seconds_*
spring.ai.advisor	spring_ai_advisor_seconds_*
spring.ai.tool: span execute_tool <name>	(was tool_call <name>)

management.endpoints.web.exposure.include=health,metrics,prometheus management.metrics.distribution.percentiles-histogram.gen_ai.client.operation=true spring.ai.chat.observations.log-prompt=false spring.ai.chat.observations.log-completion=false

- gen_ai.operation.name=execute_tool, new spring.ai.tool.type; active gauges end in _active_seconds_count
- Filter gen_ai_token_type!="total" in sums; prompt/completion logging is off by default (Pll)
- Unit tests: register ChatModelMeterObservationHandler by hand; actuator/prometheus is 401 under default Security

TESTING & TESTCONTAINERS

@TestConfiguration(proxyBeanMethods = false) class DevServices { @Bean @ServiceConnection OllamaContainer ollama() { return new OllamaContainer("ollama/ollama:0.34.4"); } @Bean @ServiceConnection PostgreSQLContainer pg() { /* pgvector/pgvector:pg17 */ } } SpringApplication.from(App::main) .with(DevServices.class).run(args);

- Test deps: spring-boot-testcontainers, spring-ai-spring-boot-testcontainers, org.testcontainers:testcontainers-ollama, testcontainers-postgresql (TC 2.x names)
- Pull models with spring.ai.ollama.init.pull-model-strategy=when_missing
- Stub ChatModel must override getOptions() returning ToolCallingChatOptions or the tool loop is skipped; WireMock (wiremock-standalone) for provider HTTP; keyless runs: spring.ai.model.moderation=none

1.X → 2.X RENAMES

1.x	2.0
ToolCallAdvisor	ToolCallingAdvisor
FunctionCallback, defaultFunctions, .functions(), FunctionCallingOptions	ToolCallback, defaultTools, .tools(), ToolCallingChatOptions
InternalToolExecutionEnabled, toolNames()	removed (loop is in the advisor)
SpeechModel family	TextToSpeechModel family
spring-ai-advisors-vector-store	spring-ai-vector-store-advisor
getDefaultOptions(), mutate(); N() → n()	getOptions(), mutate(); N() → n()
.options. properties	flattened
McpSyncClientCustomizer	McpClientCustomizer
org.springframework.ai.mcp-annotations	spring-ai-mcp-annotations
PromptChatMemoryAdvisor; memory order +1000	MessageChatMemoryAdvisor; +200
Tool span tool_call	execute_tool
com.fasterxml.jackson, McpJsonParser	tools, jackson, JsonHelper

Removed modules: spring-ai-openai-sdk, spring-ai-oci-genai, Azure OpenAI starter, MiniMax, Cosmos DB. streamToolCallResponses gone. OpenRewrite: community io.arconia.rewrite.spring.ai2.UgradeSpringAi_2_0.